1.2 Classes and Objects

In more complex Java programs, the primary "actors" are ***objects***. Every object is an ***instance*** of a class, which serves as the ***type*** of the object and as a blueprint, defining the data which the object stores and the methods for accessing and modifying that data. The critical ***members*** of a class in Java are the following:

- ***Instance variables***, which are also called ***fields***, represent the data associated with an object of a class. Instance variables must have a ***type***, which can either be a base type (such as **int**, **float**, or **double**) or any class type (also known as a ***reference type*** for reasons we soon explain).
- ***Methods*** in Java are blocks of code that can be called to perform actions (similar to functions and procedures in other high-level languages). Methods can accept parameters as arguments, and their behavior may depend on the object upon which they are invoked and the values of any parameters that are passed. A method that returns information to the caller without changing any instance variables is known as an ***accessor method***, while an ***update method*** is one that may change one or more instance variables when called.

For the purpose of illustration, Code Fragment 1.2 provides a complete definition of a very simple class named Counter, to which we will refer during the remainder of this section.

```
1  public class Counter {
2    private int count;                                  // a simple integer instance variable
3    public Counter( ) { }                               // default constructor (count is 0)
4    public Counter(int initial) { count = initial; }    // an alternate constructor
5    public int getCount( ) { return count; }            // an accessor method
6    public void increment( ) { count++; }               // an update method
7    public void increment(int delta) { count += delta; }  // an update method
8    public void reset( ) { count = 0; }                 // an update method
9  }
```

Code Fragment 1.2: A Counter class for a simple counter, which can be queried, incremented, and reset.

This class includes one instance variable, named count, which is declared at line 2. As noted on the previous page, the count will have a default value of zero, unless we otherwise initialize it.

The class includes two special methods known as constructors (lines 3 and 4), one accessor method (line 5), and three update methods (lines 6–8). Unlike the original Universe class from page 2, our Counter class does not have a main method, and so it cannot be run as a complete program. Instead, the purpose of the Counter class is to create instances that might be used as part of a larger program.

1.2.1 Creating and Using Objects

Before we explore the intricacies of the syntax for our Counter class definition, we prefer to describe how Counter instances can be created and used. To this end, Code Fragment 1.3 presents a new class named CounterDemo.

```
1  public class CounterDemo {
2    public static void main(String[ ] args) {
3      Counter c;                    // declares a variable; no counter yet constructed
4      c = new Counter( );           // constructs a counter; assigns its reference to c
5      c.increment( );               // increases its value by one
6      c.increment(3);               // increases its value by three more
7      int temp = c.getCount( );     // will be 4
8      c.reset( );                   // value becomes 0
9      Counter d = new Counter(5);// declares and constructs a counter having value 5
10     d.increment( );               // value becomes 6
11     Counter e = d;                // assigns e to reference the same object as d
12     temp = e.getCount( );         // will be 6 (as e and d reference the same counter)
13     e.increment(2);               // value of e (also known as d) becomes 8
14   }
15 }
```

Code Fragment 1.3: A demonstration of the use of Counter instances.

There is an important distinction in Java between the treatment of base-type variables and class-type variables. At line 3 of our demonstration, a new variable c is declared with the syntax:

```
Counter c;
```

This establishes the identifier, c, as a variable of type Counter, but it does not create a Counter instance. Classes are known as ***reference types*** in Java, and a variable of that type (such as c in our example) is known as a ***reference variable***. A reference variable is capable of storing the location (i.e., ***memory address***) of an object from the declared class. So we might assign it to reference an existing instance or a newly constructed instance. A reference variable can also store a special value, **null**, that represents the lack of an object.

In Java, a new object is created by using the **new** operator followed by a call to a constructor for the desired class; a constructor is a method that always shares the same name as its class. The **new** operator returns a ***reference*** to the newly created instance; the returned reference is typically assigned to a variable for further use.

In Code Fragment 1.3, a new Counter is constructed at line 4, with its reference assigned to the variable c. That relies on a form of the constructor, Counter( ), that takes no arguments between the parentheses. (Such a zero-parameter constructor is known as a ***default constructor***.) At line 9 we construct another counter using a one-parameter form that allows us to specify a nonzero initial value for the counter.

Three events occur as part of the creation of a new instance of a class:

- A new object is dynamically allocated in memory, and all instance variables are initialized to standard default values. The default values are **null** for reference variables and 0 for all base types except **boolean** variables (which are **false** by default).
- The constructor for the new object is called with the parameters specified. The constructor may assign more meaningful values to any of the instance variables, and perform any additional computations that must be done due to the creation of this object.
- After the constructor returns, the **new** operator returns a reference (that is, a memory address) to the newly created object. If the expression is in the form of an assignment statement, then this address is stored in the object variable, so the object variable ***refers*** to this newly created object.

The Dot Operator

One of the primary uses of an object reference variable is to access the members of the class for this object, an instance of its class. That is, an object reference variable is useful for accessing the methods and instance variables associated with an object. This access is performed with the dot (".") operator. We call a method associated with an object by using the reference variable name, following that by the dot operator and then the method name and its parameters. For example, in Code Fragment 1.3, we call c.increment( ) at line 5, c.increment(3) at line 6, c.getCount( ) at line 7, and c.reset( ) at line 8. If the dot operator is used on a reference that is currently **null**, the Java runtime environment will throw a NullPointerException.

If there are several methods with this same name defined for a class, then the Java runtime system uses the one that matches the actual number of parameters sent as arguments, as well as their respective types. For example, our Counter class supports two methods named increment: a zero-parameter form and a one-parameter form. Java determines which version to call when evaluating commands such as c.increment( ) versus c.increment(3). A method's name combined with the number and types of its parameters is called a method's ***signature***, for it takes all of these parts to determine the actual method to perform for a certain method call. Note, however, that the signature of a method in Java does not include the type that the method returns, so Java does not allow two methods with the same signature to return different types.

A reference variable v can be viewed as a "pointer" to some object o. It is as if the variable is a holder for a remote control that can be used to control the newly created object (the device). That is, the variable has a way of pointing at the object and asking it to do things or give us access to its data. We illustrate this concept in Figure 1.2. Using the remote control analogy, a **null** reference is a remote control holder that is empty.

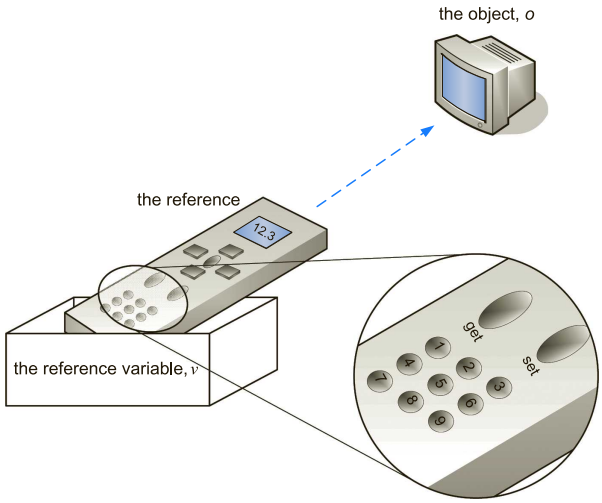

Figure 1.2: Illustrating the relationship between objects and object reference variables. When we assign an object reference (that is, memory address) to a reference variable, it is as if we are storing that object's remote control at that variable.

There can, in fact, be many references to the same object, and each reference to a specific object can be used to call methods on that object. Such a situation would correspond to our having many remote controls that all work on the same device. Any of the remotes can be used to make a change to the device (like changing a channel on a television). Note that if one remote control is used to change the device, then the (single) object pointed to by all the remotes changes. Likewise, if one object reference variable is used to change the state of the object, then its state changes for all the references to it. This behavior comes from the fact that there are many references, but they all point to the same object.

Returning to our CounterDemo example, the instance constructed at line 9 as

```
Counter d = new Counter(5);
```

is a distinct instance from the one identified as c. However, the command at line 11,

```
Counter e = d;
```

does not result in the construction of a new Counter instance. This declares a new *reference variable* named e, and assigns that variable a reference to the existing counter instance currently identified as d. At that point, both variables d and e are aliases for the same object, and so the call to d.getCount( ) behaves just as would e.getCount( ). Similarly, the call to update method e.increment(2) is affecting the same object identified by d.

It is worth noting, however, that the aliasing of two reference variables to the same object is not permanent. At any point in time, we may reassign a reference variable to a new instance, to a different existing instance, or to **null**.

1.2.2 Defining a Class

Thus far, we have provided definitions for two simple classes: the Universe class on page 2 and the Counter class on page 5. At its core, a class definition is a block of code, delimited by braces "{" and "}" , within which is included declarations of instance variables and methods that are the members of the class. In this section, we will undertake a deeper examination of class definitions in Java.

Modifiers

Immediately before the definition of a class, instance variable, or method in Java, keywords known as ***modifiers*** can be placed to convey additional stipulations about that definition.

Access Control Modifiers

The first set of modifiers we discuss are known as ***access control modifiers***, as they control the level of access (also known as ***visibility***) that the defining class grants to other classes in the context of a larger Java program. The ability to limit access among classes supports a key principle of object-orientation known as encapsulation (see Section 2.1). In general, the different access control modifiers and their meaning are as follows:

- The **public** class modifier designates that all classes may access the defined aspect. For example, line 1 of of Code Fragment 1.2 designates

  ```
  public class Counter {
  ```

  and therefore all other classes (such as CounterDemo) are allowed to construct new instances of the Counter class, as well as to declare variables and parameters of type Counter. In Java, each public class must be defined in a separate file named *classname*`.java`, where "*classname*" is the name of the class (for example, file `Counter.java` for the Counter class definition).

  The designation of **public** access for a particular *method* of a class allows any other class to make a call to that method. For example, line 5 of Code Fragment 1.2 designates

  ```
  public int getCount( ) { return count; }
  ```

  which is why the CounterDemo class may call c.getCount( ).

  If an instance variable is declared as public, dot notation can be used to directly access the variable by code in any other class that possesses a reference to an instance of this class. For example, were the count variable of Counter to be declared as public (which it is not), then the CounterDemo would be allowed to read or modify that variable using a syntax such as c.count.

- The **protected** class modifier designates that access to the defined aspect is only granted to the following groups of other classes:
  - Classes that are designated as ***subclasses*** of the given class through inheritance. (We will discuss inheritance as the focus of Section 2.2.)
  - Classes that belong to the same ***package*** as the given class. (We will discuss packages within Section 1.8.)
- The **private** class modifier designates that access to a defined member of a class be granted only to code within that class. Neither subclasses nor any other classes have access to such members.

  For example, we defined the count instance variable of the Counter class to have private access level. We were allowed to read or edit its value from within methods of that class (such as getCount, increment, and reset), but other classes such as CounterDemo cannot directly access that field. Of course, we did provide other public methods to grant outside classes with behaviors that depended on the current count value.
- Finally, we note that if no explicit access control modifier is given, the defined aspect has what is known as ***package-private*** access level. This allows other classes in the same package (see Section 1.8) to have access, but not any classes or subclasses from other packages.

The **static** Modifier

The **static** modifier in Java can be declared for any variable or method of a class (or for a nested class, as we will introduce in Section 2.6).

When a variable of a class is declared as **static**, its value is associated with the class as a whole, rather than with each individual instance of that class. Static variables are used to store "global" information about a class. (For example, a static variable could be used to maintain the total number of instances of that class that have been created.) Static variables exist even if no instance of their class exists.

When a method of a class is declared as **static**, it too is associated with the class itself, and not with a particular instance of the class. That means that the method is not invoked on a particular instance of the class using the traditional dot notation. Instead, it is typically invoked using the name of the class as a qualifier.

As an example, in the java.lang package, which is part of the standard Java distribution, there is a Math class that provides many static methods, including one named sqrt that computes square roots of numbers. To compute a square root, you do not need to create an instance of the Math class; that method is called using a syntax such as Math.sqrt(2), with the class name Math as the qualifier before the dot operator.

Static methods can be useful for providing utility behaviors related to a class that need not rely on the state of any particular instance of that class.

The **abstract** Modifier

A method of a class may be declared as **abstract**, in which case its signature is provided but without an implementation of the method body. Abstract methods are an advanced feature of object-oriented programming to be combined with inheritance, and the focus of Section 2.3.3. In short, any subclass of a class with abstract methods is expected to provide a concrete implementation for each abstract method.

A class with one or more abstract methods must also be formally declared as **abstract**, because it is essentially incomplete. (It is also permissible to declare a class as abstract even if it does not contain any abstract methods.) As a result, Java will not allow any instances of an abstract class to be constructed, although reference variables may be declared with an abstract type.

The **final** Modifier

A variable that is declared with the **final** modifier can be initialized as part of that declaration, but can never again be assigned a new value. If it is a base type, then it is a constant. If a reference variable is **final**, then it will always refer to the same object (even if that object changes its internal state). If a member variable of a class is declared as **final**, it will typically be declared as **static** as well, because it would be unnecessarily wasteful to have every instance store the identical value when that value can be shared by the entire class.

Designating a method or an entire class as **final** has a completely different consequence, only relevant in the context of inheritance. A final method cannot be overridden by a subclass, and a final class cannot even be subclassed.

Declaring Instance Variables

When defining a class, we can declare any number of instance variables. An important principle of object-orientation is that each instance of a class maintains its own individual set of instance variables (that is, in fact, why they are called *instance* variables). So in the case of the Counter class, each instance will store its own (independent) value of count.

The general syntax for declaring one or more instance variables of a class is as follows (with optional portions bracketed):

[*modifiers*] *type identifier*$_1$[=*initialValue*$_1$], *identifier*$_2$[=*initialValue*$_2$];

In the case of the Counter class, we declared

```
private int count;
```

where **private** is the modifier, **int** is the type, and count is the identifier. Because we did not declare an initial value, it automatically receives the default of zero as a base-type integer.

Declaring Methods

A method definition has two parts: the ***signature***, which defines the name and parameters for a method, and the ***body***, which defines what the method does. The method signature specifies how the method is called, and the method body specifies what the object will do when it is called. The syntax for defining a method is as follows:

[*modifiers*] *returnType methodName*(*type*$_1$ *param*$_1$, ..., *type*$_n$ *param*$_n$) {
    // method body . . .
}

Each of the pieces of this declaration has an important purpose. We have already discussed the significance of *modifiers* such as **public**, **private**, and **static**. The *returnType* designation defines the type of value returned by the method. The *methodName* can be any valid Java identifier. The list of parameters and their types declares the local variables that correspond to the values that are to be passed as arguments to this method. Each type declaration *type*$_i$ can be any Java type name and each *param*$_i$ can be any distinct Java identifier. This list of parameters and their types can be empty, which signifies that there are no values to be passed to this method when it is invoked. These parameter variables, as well as the instance variables of the class, can be used inside the body of the method. Likewise, other methods of this class can be called from inside the body of a method.

When a (nonstatic) method of a class is called, it is invoked on a specific instance of that class and can change the state of that object. For example, the following method of the Counter class increases the counter's value by the given amount.

```
public void increment(int delta) {
  count += delta;
}
```

Notice that the body of this method uses count, which is an instance variable, and delta, which is a parameter.

Return Types

A method definition must specify the type of value the method will return. If the method does not return a value (as with the increment method of the Counter class), then the keyword **void** must be used. To return a value in Java, the body of the method must use the **return** keyword, followed by a value of the appropriate return type. Here is an example of a method (from the Counter class) with a nonvoid return type:

```
public int getCount( ) {
  return count;
}
```

Java methods can return only one value. To return multiple values in Java, we should instead combine all the values we want to return in a ***compound object***, whose instance variables include all the values we want to return, and then return a reference to that compound object. In addition, we can change the internal state of an object that is passed to a method as another way of "returning" multiple results.

Parameters

A method's parameters are defined in a comma-separated list enclosed in parentheses after the name of the method. A parameter consists of two parts, the parameter type and the parameter name. If a method has no parameters, then only an empty pair of parentheses is used.

All parameters in Java are passed ***by value***, that is, any time we pass a parameter to a method, a copy of that parameter is made for use within the method body. So if we pass an **int** variable to a method, then that variable's integer value is copied. The method can change the copy but not the original. If we pass an object reference as a parameter to a method, then the reference is copied as well. Remember that we can have many different variables that all refer to the same object. Reassigning the internal reference variable inside a method will not change the reference that was passed in.

For the sake of demonstration, we will assume that the following two methods were added to an arbitrary class (such as CounterDemo).

```
public static void badReset(Counter c) {
  c = new Counter( );              // reassigns local name c to a new counter
}

public static void goodReset(Counter c) {
  c.reset( );                      // resets the counter sent by the caller
}
```

Now we will assume that variable strikes refers to an existing Counter instance in some context, and that it currently has a value of 3.

If we were to call badReset(strikes), this has *no* effect on the Counter known as strikes. The body of the badReset method reassigns the (local) parameter variable c to reference a newly created Counter instance; but this does not change the state of the existing counter that was sent by the caller (i.e., strikes).

In contrast, if we were to call goodReset(strikes), this does indeed reset the caller's counter back to a value of zero. That is because the variables c and strikes are both reference variables that refer to the same Counter instance. So when c.reset( ) is called, that is effectively the same as if strikes.reset( ) were called.

Defining Constructors

A ***constructor*** is a special kind of method that is used to initialize a newly created instance of the class so that it will be in a consistent and stable initial state. This is typically achieved by initializing each instance variable of the object (unless the default value will suffice), although a constructor can perform more complex computation. The general syntax for declaring a constructor in Java is as follows:

modifiers *name*(*type*$_0$ *parameter*$_0$, ..., *type*$_{n-1}$ *parameter*$_{n-1}$) {
    // constructor body . . .
}

Constructors are defined in a very similar way as other methods of a class, but there are a few important distinctions:

1. Constructors cannot be **static**, **abstract**, or **final**, so the only modifiers that are allowed are those that affect visibility (i.e., **public**, **protected**, **private**, or the default package-level visibility).
2. The name of the constructor must be identical to the name of the class it constructs. For example, when defining the Counter class, a constructor must be named Counter as well.
3. We don't specify a return type for a constructor (not even **void**). Nor does the body of a constructor explicitly return anything. When a user of a class creates an instance using a syntax such as

   ```
   Counter d = new Counter(5);
   ```

   the **new** operator is responsible for returning a reference to the new instance to the caller; the responsibility of the constructor method is only to initialize the state of the new instance.

A class can have many constructors, but each must have a different ***signature***, that is, each must be distinguished by the type and number of the parameters it takes. If no constructors are explicitly defined, Java provides an implicit ***default constructor*** for the class, having zero arguments and leaving all instance variables initialized to their default values. However, if a class defines one or more nondefault constructors, no default constructor will be provided.

As an example, our Counter class defines the following pair of constructors:

```
public Counter( ) { }
public Counter(int initial) { count = initial; }
```

The first of these has a trivial body, { }, as the goal for this default constructor is to create a counter with value zero, and that is already the default value of the integer instance variable, count. However, it is still important that we declared such an explicit constructor, because otherwise none would have been provided, given the existence of the nondefault constructor. In that scenario, a user would have been unable to use the syntax, **new** Counter( ).

The Keyword **this**

Within the body of a (nonstatic) method in Java, the keyword **this** is automatically defined as a reference to the instance upon which the method was invoked. That is, if a caller uses a syntax such as thing.foo(a, b, c), then within the body of method foo for that call, the keyword **this** refers to the object known as thing in the caller's context. There are three common reasons why this reference is needed from within a method body:

1. To store the reference in a variable, or send it as a parameter to another method that expects an instance of that type as an argument.

2. To differentiate between an instance variable and a local variable with the same name. If a local variable is declared in a method having the same name as an instance variable for the class, that name will refer to the local variable within that method body. (We say that the local variable *masks* the instance variable.) In this case, the instance variable can still be accessed by explicitly using the dot notation with **this** as the qualifier. For example, some programmers prefer to use the following style for a constructor, with a parameter having the same name as the underlying variable.

```
public Counter(int count) {
  this.count = count;   // set the instance variable equal to parameter
}
```

3. To allow one constructor body to invoke another constructor body. When one method of a class invokes another method of that same class on the current instance, that is typically done by using the (unqualified) name of the other method. But the syntax for calling a constructor is special. Java allows use of the keyword **this** to be used as a method within the body of one constructor, so as to invoke another constructor with a different signature.

   This is often useful because all of the initialization steps of one constructor can be reused with appropriate parameterization. As a trivial demonstration of the syntax, we could reimplement the zero-argument version of our Counter constructor to have it invoke the one-argument version sending 0 as an explicit parameter. This would be written as follows:

```
public Counter( ) {
  this(0);          // invoke one-parameter constructor with value zero
}
```

   We will provide a more meaningful demonstration of this technique in a later example of a CreditCard class in Section 1.7.

The main Method

Some Java classes, such as our Counter class, are meant to be used by other classes, but are not intended to serve as a self-standing program. The primary control for an application in Java must begin in some class with the execution of a special method named main. This method must be declared as follows:

```
public static void main(String[ ] args) {
  // main method body...
}
```

The args parameter is an array of String objects, that is, a collection of indexed strings, with the first string being args[0], the second being args[1], and so on. (We say more about strings and arrays in Section 1.3.) Those represent what are known as ***command-line arguments*** that are given by a user when the program is executed.

Java programs can be called from the command line using the `java` command (in a Windows, Linux, or Unix shell), followed by the name of the Java class whose main method we want to run, plus any optional arguments. For example, to execute the main method of a class named Aquarium, we could issue the following command:

```
java Aquarium
```

In this case, the Java runtime system looks for a compiled version of the Aquarium class, and then invokes the special main method in that class.

If we had defined the Aquarium program to take an optional argument that specifies the number of fish in the aquarium, then we might invoke the program by typing the following in a shell window:

```
java Aquarium 45
```

to specify that we want an aquarium with 45 fish in it. In this case, args[0] would refer to the string `"45"`. It would be up to the body of the main method to interpret that string as the desired number of fish.

Programmers who use an integrated development environment (IDE), such as Eclipse, can optionally specify command-line arguments when executing the program through the IDE.

Unit Testing

When defining a class, such as Counter, that is meant to be used by other classes rather than as a self-standing program, there is no need to define a main method. However, a nice feature of Java's design is that we could provide such a method as a way to test the functionality of that class in isolation, knowing that it would not be run unless we specifically invoke the `java` command on that isolated class. However, for more robust testing, frameworks such as JUnit are preferred.